

```

#include <stdint.h>
#include "tm4c1294ncpdt.h"
#include "SysTick.h"
#include "PLL.h"

// GLOBAL VARIABLE for Keil Debugger (Watch 1)
// This will store the 8-bit code (PM[3:0] PE[3:0]) requested in Milestone 7.1
volatile uint8_t scan_code = 0b11111111;

volatile uint8_t scan_8;

void light(uint8_t scan_8)
{
    GPIO_PORTN_DATA_R=0b00000000;
    GPIO_PORTF_DATA_R=0b00000000;

    if(scan_8 & 0b00000010)
        GPIO_PORTN_DATA_R|=0b00000001;
    if(scan_8 & 0b00000001)
        GPIO_PORTN_DATA_R|=0b00000010;
    if(scan_8 & 0b00001000)
        GPIO_PORTF_DATA_R|=0b00000001;
    if(scan_8 & 0b00000100)
        GPIO_PORTF_DATA_R|=0b00010000;
}

int H_to_B(uint8_t scan_code){
    if (scan_code == 0xEE)        //^1
        scan_8 = 1;
    if (scan_code == 0xED)//2
        scan_8 = 2;
    if (scan_code==0xEB)//3
        scan_8 = 3;
    if (scan_code==0xDE)//4
        scan_8 = 4;
    if (scan_code==0xDD)//5
        scan_8 = 5;
    if (scan_code==0xDB)//6
        scan_8 = 6;
    if (scan_code==0xBE)//7
        scan_8 = 7;
    if (scan_code==0xBD)//8
        scan_8 = 8;
}

```

```

        if (scan_code==0xBB)//9
            scan_8 = 9;
        if (scan_code==0x7D)//0
            scan_8 = 0;
        if (scan_code==0xE7)//A
            scan_8 = 10;
        if (scan_code==0xD7)//B
            scan_8 = 11;
        if (scan_code==0xB7)//C
            scan_8 = 12;
        if (scan_code==0x77)//D
            scan_8 = 13;
        if (scan_code==0x7E)//*
            scan_8 = 14;
        if (scan_code==0x7B)//#
            scan_8 = 15;

        return scan_8;
    }

void PortE_Init(void){
    SYSCTL_RCGCGPIO_R |= SYSCTL_RCGCGPIO_R4;           // Activate the clock for
Port E
    while((SYSCTL_PRGPIO_R & SYSCTL_PRGPIO_R4) == 0){}; // Allow time for clock to
stabilize

    GPIO_PORTE_DIR_R = 0b00001111;                     // Enable PE0-PE3 as OUTPUTS
(Rows)
    GPIO_PORTE_DEN_R = 0b00001111;                     // Enable PE0-PE3 as digital pins
}

void PortM_Init(void){
    SYSCTL_RCGCGPIO_R |= SYSCTL_RCGCGPIO_R11;          // Activate the clock for
Port M
    while((SYSCTL_PRGPIO_R & SYSCTL_PRGPIO_R11) == 0){}; // Allow time for clock to
stabilize

    GPIO_PORTM_DIR_R = 0b00000000;                     // FIXED: Enable PM0-PM3 as
INPUTS (Columns)
    GPIO_PORTM_DEN_R = 0b00001111;                     // Enable PM0-PM3 as digital pins
    GPIO_PORTM_PUR_R = 0b00001111;                     // FIXED: Enable Pull-Up resistors
on PM0-PM3
}

```

```

void PortF_Init(void)
{
    SYSCTL_RCGCGPIO_R |= SYSCTL_RCGCGPIO_R5;           // Activate the clock
for Port F
    while((SYSCTL_PRGPIO_R & SYSCTL_PRGPIO_R5) == 0){}; // Allow time for clock to
stabilize

    GPIO_PORTF_DIR_R = 0b00010001;                     // FIXED: Enable PF0-PF3 as outputs
    GPIO_PORTF_DEN_R = 0b00010001;                     // Enable PF0-PF3 as digital pins
}

```

```

void PortN_Init(void)
{
    SYSCTL_RCGCGPIO_R |= SYSCTL_RCGCGPIO_R12;          // Activate the clock
for Port N
    while((SYSCTL_PRGPIO_R & SYSCTL_PRGPIO_R12) == 0){}; // Allow time for clock to
stabilize

    GPIO_PORTN_DIR_R = 0b000000011;                    // FIXED: Enable PN0-PN as outputs
    GPIO_PORTN_DEN_R = 0b000000011;                    // Enable PN0-PN3 as digital pins
}

```

```

int main(void){
    // Initialize standard clock and ports
    PLL_Init();
    PortE_Init();
    PortM_Init();
        PortF_Init();
        PortN_Init();

        GPIO_PORTF_DATA_R = 0b000000000;
        GPIO_PORTN_DATA_R = 0b000000000;

    uint8_t current_row;
    uint8_t read_col;

    while(1){
        // Loop through the 4 rows (PE0, PE1, PE2, PE3)
        for(int i = 0; i < 4; i++) {

            // Drive ONE row LOW at a time.
            // If i=0: we drive PE0 low (1110)

```

```

// If i=1: we drive PE1 low (1101), etc.
current_row = ~(1 << i) & 0x0F;
GPIO_PORTE_DATA_R = current_row;

// Short delay to allow electrical signals to settle
for(int delay=0; delay<20; delay++);

// Read the columns from Port M
read_col = GPIO_PORTM_DATA_R & 0x0F;

// If read_col is NOT 1111 (0x0F), a button is being pressed in this row!
if(read_col != 0x0F) {

    // MILESTONE 7.1: Combine Port M (Columns) and Port E (Rows) into an 8-bit code
    // Shift Port M left by 4 bits, then OR it with Port E.
    scan_code = (read_col << 4) | current_row;
    H_to_B(scan_code);

    light(scan_8);

    // Wait here until the user RELEASES the button so we don't spam the debugger
    while((GPIO_PORTM_DATA_R & 0x0F) != 0x0F);

}

}

}

```